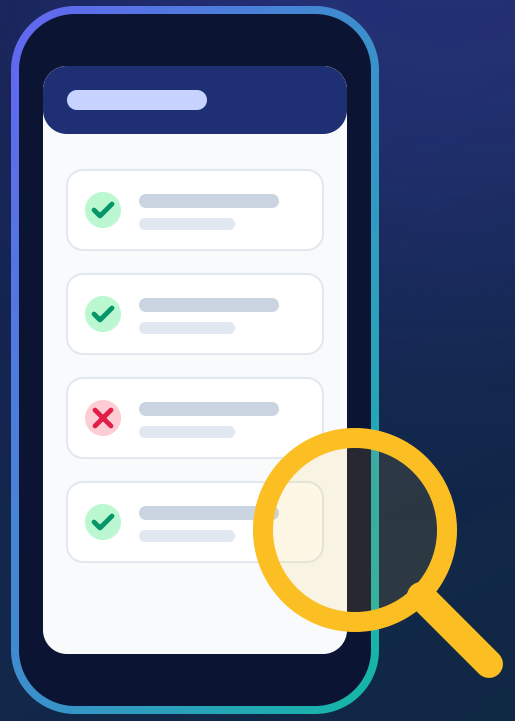
IT-WS07 · MOBILE APPLICATION DEVELOPMENT · LESSON 6

Mobile Application Usability, User Experience Evaluation, and Functional Testing Strategies



UNIT 1
Usability and User Experience Evaluation

UNIT 2
Functional Testing Strategies for Mobile Apps

Discussion-first · Live demos · Group testing

Press → to begin · ? for shortcuts · All app examples in this lesson are fictional.

Opening Discussion: It Works… But Is It Good?

DISCUSSION QUESTION

If an application works correctly but users cannot figure out how to use it, would you consider it a good application? Why or why not?

1. Think — 1 min

Write your answer privately (notes or chat draft).

2. Pair — 2 min

Compare with a partner in a breakout room or DM.

3. Share — 2–3 volunteers

Unmute or post your pair's answer in the chat.

Picture this (fictional)

The school portal correctly saves enrolment data — no bugs found. But the "Enrol" action is under *Menu* → *Records* → *Academic* → *Load*, and the help desk gets 300 calls in the first week.

SUGGESTED ANSWER

Not a good application overall. Functional correctness is *necessary but not sufficient*. Users judge an app by whether they can achieve their goals with reasonable effort and confidence. If they can't find or understand a feature, the value of the correct code never reaches them — and the cost moves to users, help desks and deadlines.

FOLLOW-UP QUESTIONS

- → Who pays the cost when users cannot figure it out?
- → Is an app that is easy to use but sometimes wrong any better? Which is worse for a grades portal?

UI, UX, Usability, Accessibility, and Functional Correctness

Same feature, five different questions — example: uploading a profile picture

UI
USER INTERFACE

What does the user see and touch?

Screens, controls, layout, icons, typography, colour.

The camera icon and "Save photo" button.

UX
USER EXPERIENCE

How does the whole experience feel?

Perceptions and responses before, during and after use — shaped by context and expectations.

Feeling confident the new photo is really saved and visible to teachers.

Usability
ISO 9241-11

Can specified users achieve goals well?

Effectiveness, efficiency and satisfaction for specified users, goals and context.

Students update their photo in under a minute without help.

Accessibility
E.G., WCAG 2.2

Can people with diverse abilities use it?

Perceivable, operable, understandable, robust — for people with disabilities too.

A screen-reader user hears "Change profile photo, button".

Functional correctness
VS REQUIREMENTS

Does it behave as specified?

Features produce the required results for valid and invalid inputs.

The uploaded JPG is stored and shown; a 30 MB file is rejected.

Key idea

An app can be **functionally correct** and still be **hard to use**. A beautiful UI does not guarantee good usability, and good usability is only one part of UX.

These overlap — but answer different questions

Use the question each concept asks to decide what you are evaluating.

Quick poll: The photo is saved correctly, but "Save" is hidden behind the keyboard and many students think it failed. Which concept is most directly affected?

A. Functional correctness
The photo is saved as required, so behaviour is correct.

✓ Usability
Users struggle to complete the goal efficiently and confidently.

C. Accessibility
Possibly related, but the problem affects all users here.

D. UI only
The UI causes it, but the impact is on task success — usability.

Nielsen’s 10 Usability Heuristics

Broad rules of thumb for interaction design — Jakob Nielsen, Nielsen Norman Group

1. **Visibility of System Status**
2. **Match Between the System and the Real World**
3. **User Control and Freedom**
4. **Consistency and Standards**
5. **Error Prevention**
6. **Recognition Rather Than Recall**
7. **Flexibility and Efficiency of Use**
8. **Aesthetic and Minimalist Design**
9. **Help Users Recognize, Diagnose, and Recover from Errors**
10. **Help and Documentation**

Heuristic evaluation

A few evaluators independently inspect each screen against the heuristics, list violations, then merge findings and rate severity.

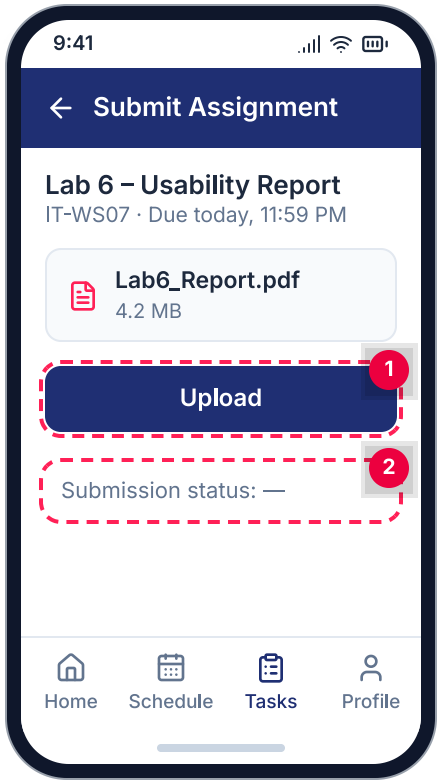
Inspection ≠ user testing

Heuristics *predict* likely problems. Testing with real users *observes* them. Use both.

Click a heuristic to jump to its slide.

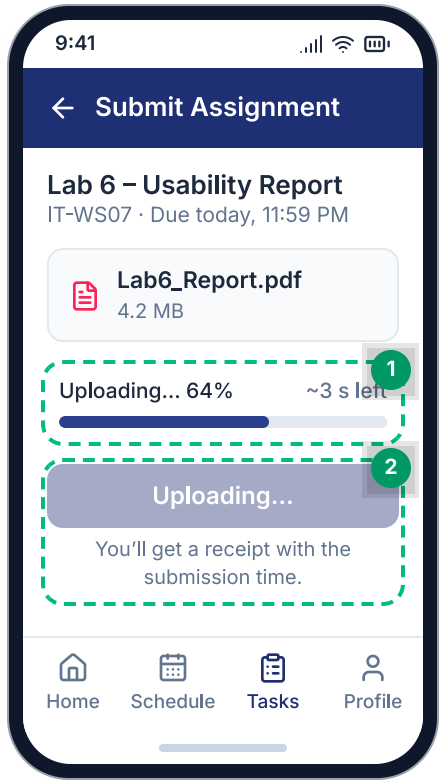
1. Visibility of System Status

✕ Problem



1. Tapped 6 s ago: no spinner, no progress, button unchanged
2. No confirmation — was it submitted?

✓ Better



1. Progress and time remaining
2. Button disabled while uploading; receipt promised

Keep users informed about what is going on, through appropriate feedback within a reasonable amount of time.

DISCUSS

A student taps Upload and nothing visibly happens for 8 seconds. What will most students do next, and why is that a problem?

INSTRUCTOR EXPLANATION

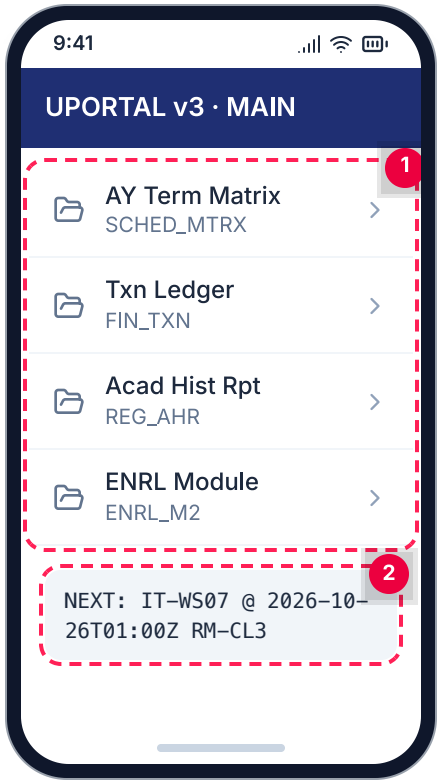
Most will **tap again**, leave the screen, or assume it failed. That can create duplicate submissions, cancel the upload, or cause panic near a deadline. The system knows it is working — it just isn't telling the user. Feedback should be immediate (button state), continuous (progress), and conclusive (confirmation with time stamp).

Possible design improvement

Show a progress indicator, disable the button during upload, and confirm success with a time-stamped receipt.

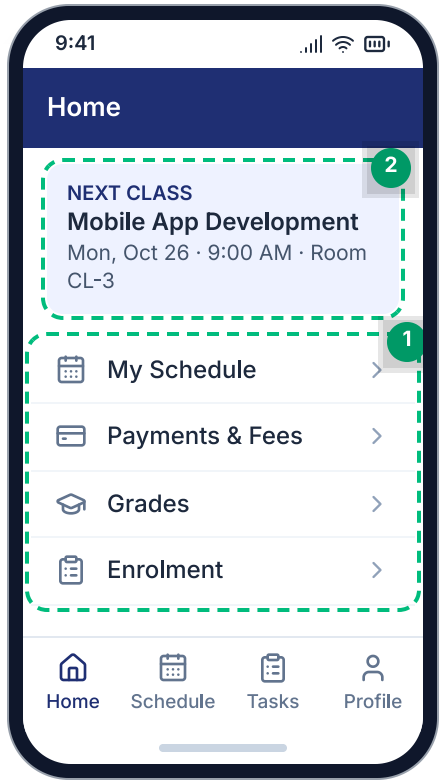
2. Match Between the System and the Real World

✕ Problem



1. Internal system names instead of student words
2. Machine timestamp (in UTC) instead of a readable local date

✓ Better



1. Labels students would use themselves
2. Readable local date, time and room

Speak the users' language — words, phrases and concepts familiar to them — and follow real-world conventions.

DISCUSS

Which words on the left screen would a first-year student not understand? Whose language is it?

INSTRUCTOR EXPLANATION

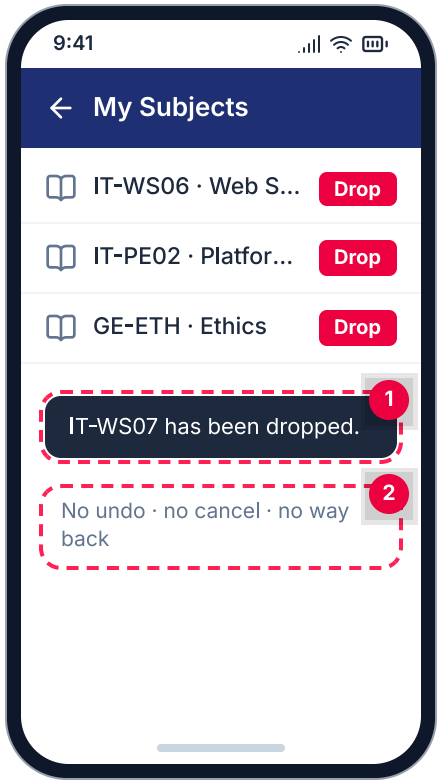
"AY Term Matrix," "Txn Ledger," "Acad Hist Rpt," the module codes and the UTC timestamp. This is the **developers' and registrar's internal language**. Students think in terms of "my schedule," "fees," and "grades." Verify labels with real users (e.g., quick task tests or card sorting) — don't assume your vocabulary is theirs.

Possible design improvement

Rename items using students' words, add familiar icons, and display dates in local, human-readable format.

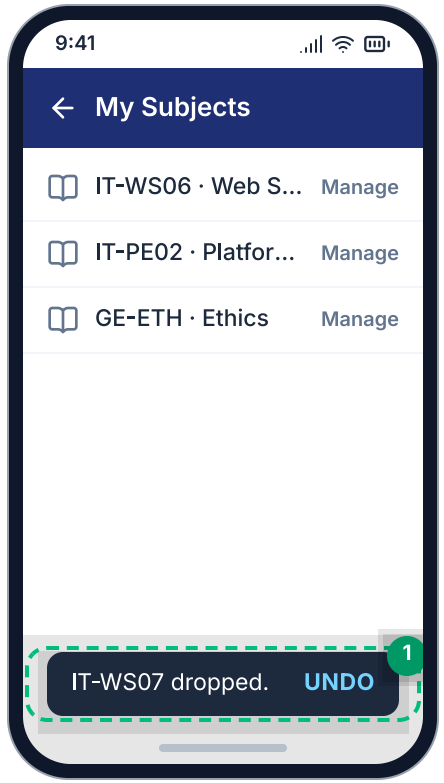
3. User Control and Freedom

✕ Problem



1. One tap drops a subject instantly
2. No undo or confirmation for a high-impact action

✓ Better



1. Undo restores the subject immediately
2. "Manage" moves Drop off the main list, away from accidental taps

Users often perform actions by mistake. They need a clearly marked "emergency exit" to leave the unwanted action — undo, cancel, back.

DISCUSS

A student accidentally drops a subject on the left screen. What should the app let them do?

INSTRUCTOR EXPLANATION

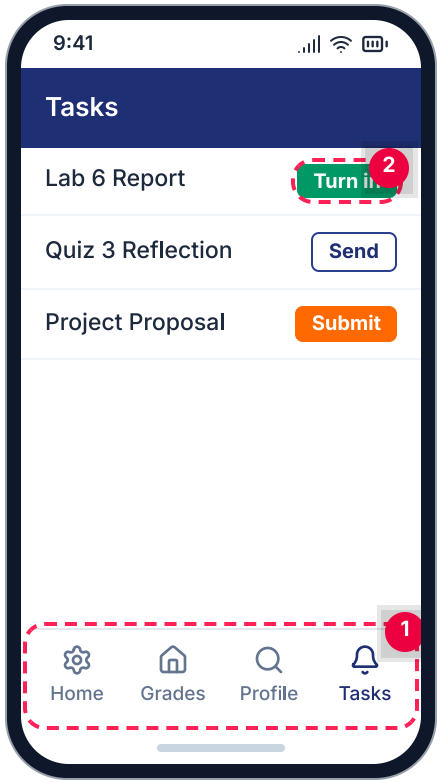
Reverse it immediately — an **Undo** option — and make destructive actions harder to trigger by accident. For truly irreversible, high-cost actions, a clear confirmation explaining the consequence is appropriate. Every flow should also have an obvious **Back/Cancel** that does not lose the user's work.

Possible design improvement

Provide Undo for reversible actions, confirm irreversible ones, and keep Back/Cancel visible in every flow.

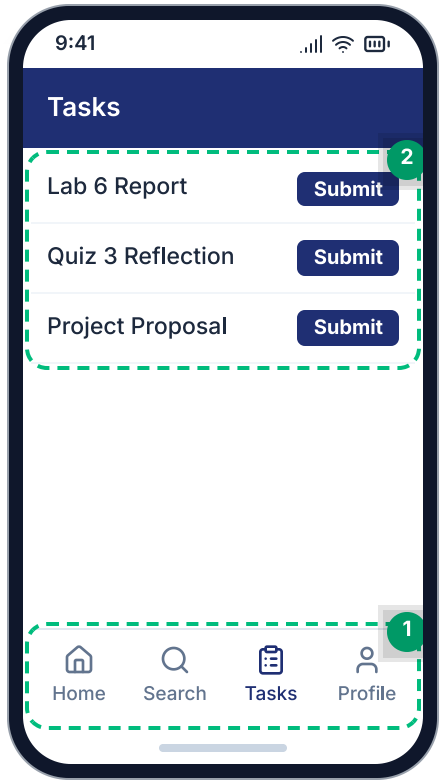
4. Consistency and Standards

✕ Problem



1. Familiar icons with different meanings (house = Grades, magnifier = Profile)
2. Same action named and styled three ways

✓ Better



1. Standard icons with their usual meaning
2. One label and one style for the same action

Users should not have to wonder whether different words, situations or actions mean the same thing. Follow platform and industry conventions.

DISCUSS

Why is reusing a familiar icon for a different meaning worse than using no icon at all?

INSTRUCTOR EXPLANATION

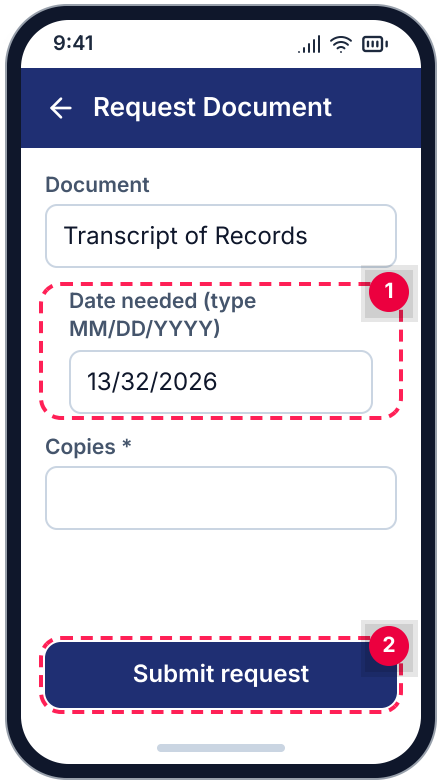
Users bring expectations from **every other app** they use (external consistency). A house icon already means “Home,” so users tap it confidently — and land somewhere unexpected. That causes errors and erodes trust. Inside the app, using one word and style for one action (internal consistency) lets users learn once and reuse that knowledge.

Possible design improvement

Use platform-standard icons and patterns, and a single consistent label and style for each action.

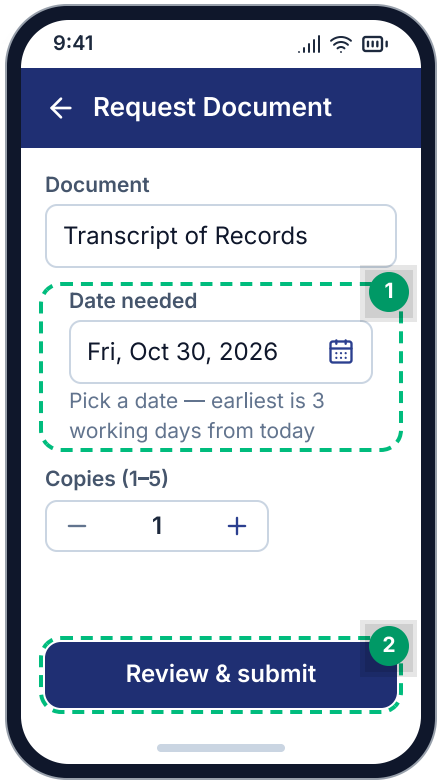
5. Error Prevention

✕ Problem



1. Free-text date accepts impossible values
2. Submit enabled with a required field empty; errors only after submitting

✓ Better



1. Date picker shows only valid dates
2. Constrained input, sensible default, review step before submitting

Good error messages are important, but the best designs prevent problems from occurring in the first place.

DISCUSS

Which is better: a great error message, or a design where the error cannot happen? Why?

INSTRUCTOR EXPLANATION

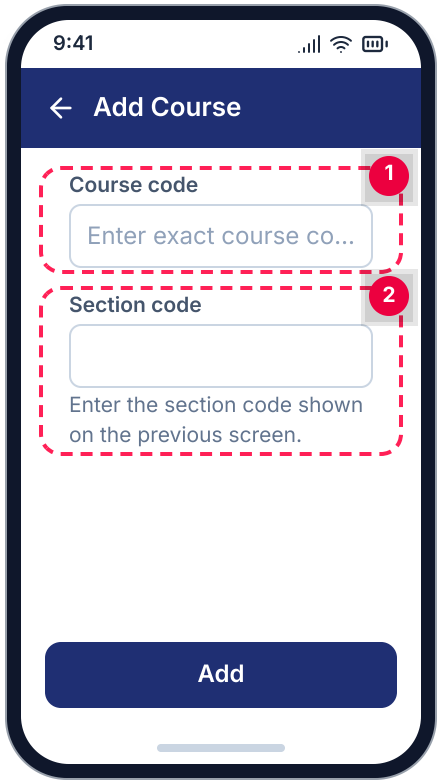
Prevention is better: every error costs the user time and confidence, even with a good message. Use **constraints** (pickers, steppers, allowed ranges), **good defaults**, and **inline hints** to prevent slips; use a review or confirmation step before costly actions. Error messages (Heuristic 9) are still needed for errors that cannot be prevented.

Possible design improvement

Replace free text with constrained controls, show limits up front, and add a review step before submitting.

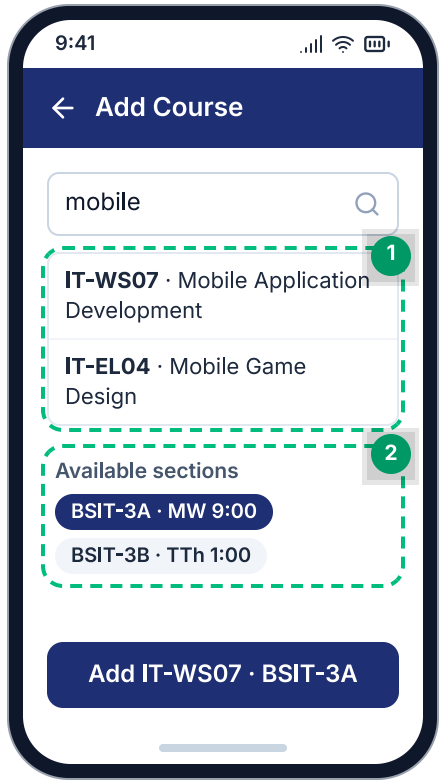
6. Recognition Rather Than Recall

✕ Problem



1. User must recall an exact code
2. Information from a previous screen must be memorised

✓ Better



1. Search suggestions with full course names
2. Sections shown as choices — nothing to remember

Minimise the user's memory load by making elements, actions and options visible. Information should not have to be remembered from one screen to another.

DISCUSS

On the left screen, what must the student remember? Where should that information come from instead?

INSTRUCTOR EXPLANATION

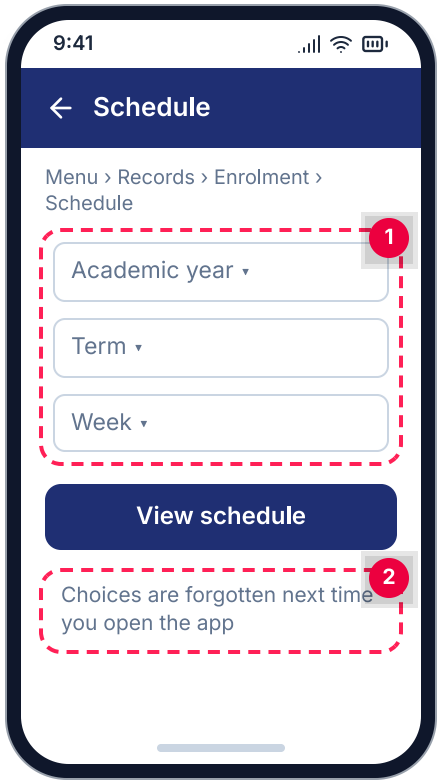
They must remember the **exact course code** and a **section code from another screen**. The app already has this information — it should **show it**: searchable suggestions, recently used items, and selectable options. Recognising "Mobile Application Development" is much easier than recalling "IT-WS07".

Possible design improvement

Offer search with suggestions and full names, show recent items, and present valid sections as choices.

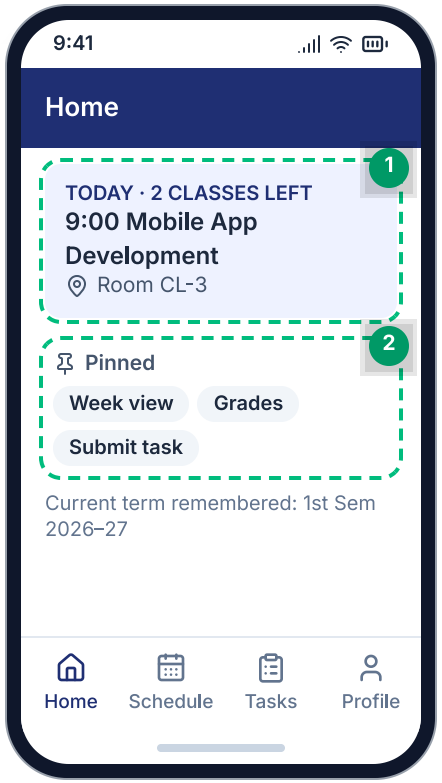
7. Flexibility and Efficiency of Use

✕ Problem



1. Four screens deep, three selections — every time
2. No remembered defaults for a daily task

✓ Better



1. Today's classes on Home — zero navigation
2. User-pinned shortcuts and remembered term

Shortcuts — hidden from novice users — may speed up interaction for experts, so the design can cater to both. Allow users to tailor frequent actions.

DISCUSS

How can an app speed up frequent users without confusing new users?

INSTRUCTOR EXPLANATION

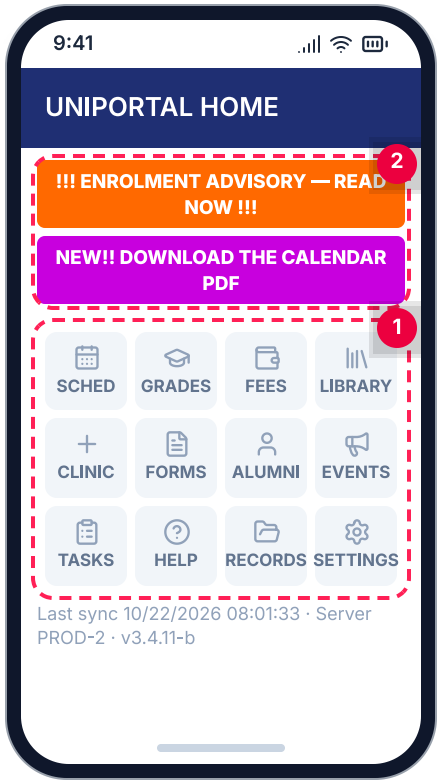
Add **accelerators** that novices can ignore: a "Today" card, pinned favourites, remembered defaults (current term), app-icon shortcuts, recent items. Keep the standard visible path for newcomers. Avoid making hidden gestures the *only* way to do something.

Possible design improvement

Surface frequent tasks on Home, remember defaults, and allow pinned shortcuts while keeping the standard path.

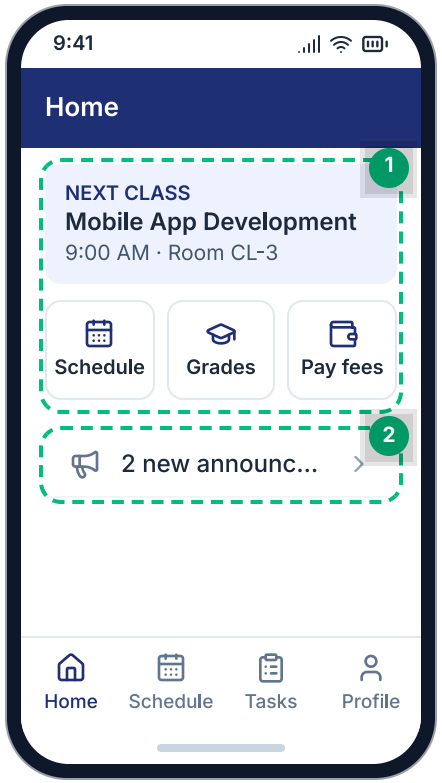
8. Aesthetic and Minimalist Design

✕ Problem



1. 12 equal-weight tiles — nothing stands out
2. Shouting banners and technical details compete for attention

✓ Better



1. Clear priority: next class plus three key actions
2. Announcements available, but not shouting

Interfaces should not contain information that is irrelevant or rarely needed. Every extra unit of information competes with the relevant units.

DISCUSS

True or false: “Minimalist design” means an app should have fewer features.

INSTRUCTOR EXPLANATION

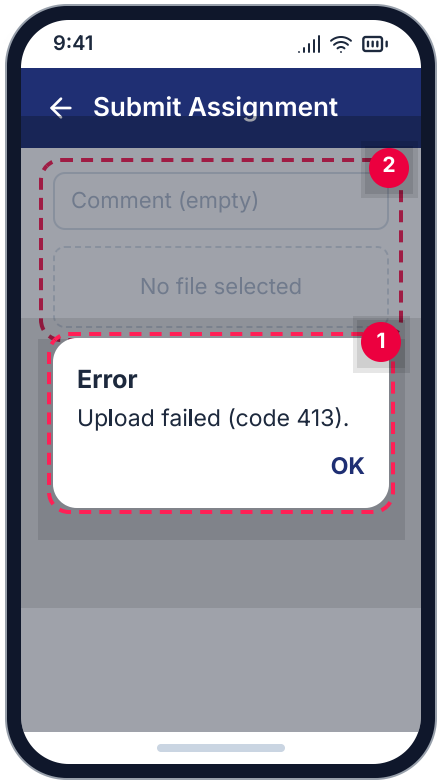
False. Minimalism is about removing *irrelevant or competing* information from a given screen so that what supports the user’s goal stands out. Features can live elsewhere (menus, search, secondary screens). It is also not a visual style — a “flat, modern” screen can still be cluttered.

Possible design improvement

Prioritise content by user goals, group secondary features, and replace shouting banners with a calm announcements entry.

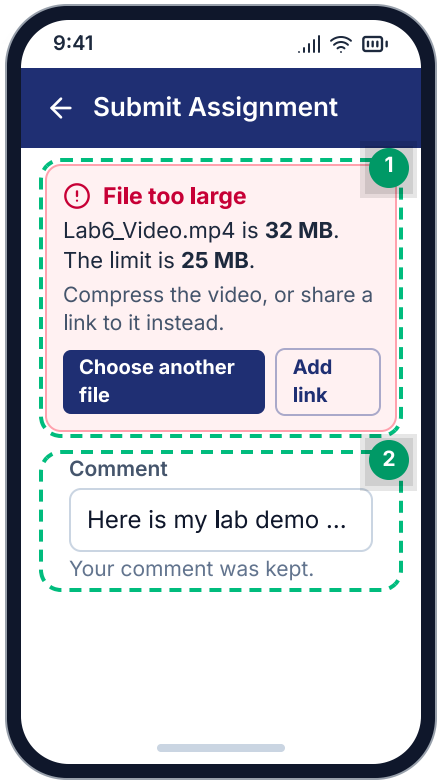
9. Help Users Recognize, Diagnose, and Recover from Errors

✕ Problem



1. Error code with no explanation or next step
2. Form was cleared — the comment the student typed is lost

✓ Better



1. Plain language: the problem, the limit, and what to do
2. User's input preserved

Error messages should be expressed in plain language (no error codes), precisely indicate the problem, and constructively suggest a solution.

DISCUSS

Rewrite "Upload failed (code 413)" so that a student can fix the problem themselves.

INSTRUCTOR EXPLANATION

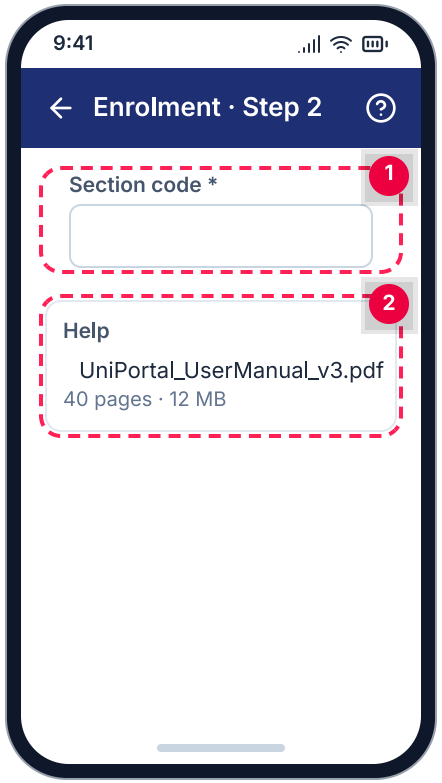
Example: *"Lab6_Video.mp4 is 32 MB, but the limit is 25 MB. Compress the video or add a link instead."* A good message is **visible near the problem**, in **plain language**, **precise** (which file, what limit), and **constructive** (a way forward), and it **keeps the user's work**. An error code can be added in small print for the help desk.

Possible design improvement

Explain the specific problem in plain language, offer recovery actions, and preserve everything the user entered.

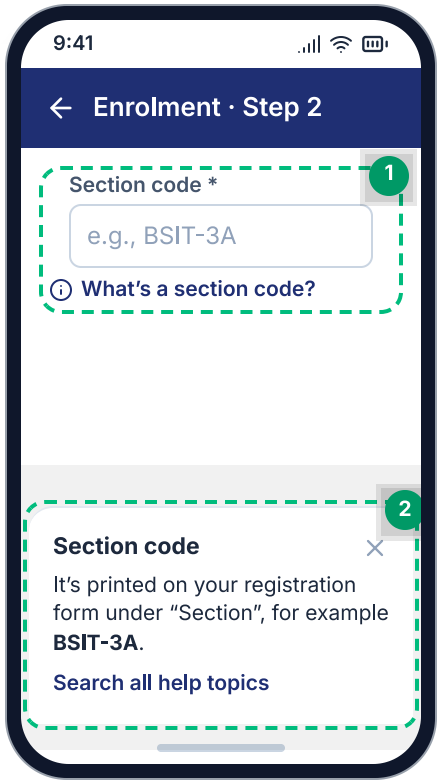
10. Help and Documentation

✕ Problem



1. Unfamiliar field with no explanation
2. Help = a long PDF manual, not searchable on a phone

✓ Better



1. Contextual help right where it's needed
2. Short, task-focused answer with a link to searchable help

It's best if the system doesn't need additional explanation. Still, documentation may be needed — it should be easy to search, focused on the user's task, and concise.

DISCUSS

If an app needs a lot of documentation, what does that suggest about its design? Should we still provide help?

INSTRUCTOR EXPLANATION

Heavy reliance on documentation often signals that the interface itself is unclear — fix the design first. But help is still needed for complex or infrequent tasks. Make it **contextual** (at the point of need), **searchable**, **task-focused** and **concise**, with concrete steps or examples.

Possible design improvement

Add short contextual help next to unfamiliar fields and a searchable, task-based help centre.